

Location Strategies: History & Current State

This document summarizes the location strategies that have been tried and which ones are currently in use for the Locate Me feature.

Overview

The Locate Me feature requires two main components:

1. **Reverse Geocoding**: Finding nearby cities from GPS coordinates
2. **Boundary Lookup**: Retrieving administrative boundary polygons (regional/local)

Reverse Geocoding Strategies

✓ **Current: GeoNames Offline Dataset**

Status: ✓ **In Use** (Primary)

Implementation:

- **Data Source:** GeoNames `cities5000` dump
- **Storage:** `data/geocoder/cities.min.json.gz` (compressed, ~5-10MB)
- **Build Script:** `scripts/geocoder/build-geocoder.js`
- **API:** `api/reverse-geocode.js`

Algorithm:

1. **Bounding Box Pre-filter:** Quick lat/lon bounding box check ($1^\circ \approx 111\text{km}$) to reduce search space
2. **Haversine Distance:** Computes exact distance for all cities within bounding box
3. **Distance Filter:** Filters out cities beyond `maxDistanceKm` threshold (default: 50km)
4. **Sorting & Selection:** Sorts by distance, prioritizes cities in same admin region (`admin1`)

Advantages:

- ✓ **Offline/On-device:** No network calls required
- ✓ **Fast:** Brute-force optimized with bounding box pre-filtering
- ✓ **Accurate:** Scans all candidates within range (not just first N)
- ✓ **Privacy-preserving:** All processing happens client-side

Used By:

- Interactive Locate Me page (`public/locate-me.html`)
- Gallery generation (`scripts/generate-locate-me-gallery.js`)
- Reverse geocoding API (`api/reverse-geocode.js`)

Boundary Lookup Strategies

× **Legacy: GADM Shapefiles (Server-Side)**

Status: × **Deprecated** (Replaced)

Implementation:

- **Data Source:** GADM shapefiles (v4.1)
- **Storage:** `gs://levante-assets-dev/maps/gadm/` (GCS bucket)
- **API:** `api/gadm-polygon.js` (legacy version)

- **Documentation:** `docs/locate-me-gadm.md`

How It Worked:

1. **Download + Cache:** On first request, downloaded zipped shapefile from GCS
2. **Snippet Index:** Pre-built index keyed by `name|admin1|country`
3. **Exact Match First:** Tried snippet match before point-in-polygon
4. **Fallback Scan:** If no snippet match, scanned all features with `@turf/boolean-point-in-polygon`

Why Replaced:

- × **Server-side dependency:** Required GCS downloads and server processing
- × **Large files:** Shapefiles were 100+ MB per country
- × **Slower:** Network latency + processing time
- × **Less granular:** GADM boundaries were less detailed than OSM

Current State:

- Legacy code preserved in `docs/locate-me-gadm.md`
- GADM shapefiles still stored in GCS (for reference)
- API endpoint (`api/gadm-polygon.js`) now uses Overpass instead

⚠ Transitional: OpenStreetMap/Overpass (Server-Side API)

Status: ⚠ **In Use** (API endpoint only, not primary flow)

Implementation:

- **Data Source:** OpenStreetMap via Overpass Turbo API
- **Endpoint:** `https://overpass-api.de/api/interpreter`
- **API:** `api/gadm-polygon.js` (current version, despite filename)
- **Caching:** 5-minute in-memory cache

How It Works:

1. **Overpass Query:** Queries OSM for admin boundaries containing GPS point (admin_level 6-10)
2. **Filtering:** Filters out counties (admin_level=6) if city/town boundaries (≥ 7) available
3. **Selection:** Chooses most specific boundary (highest admin level, then smallest area)
4. **Nearby City Fallback:** If only county found, queries nearby cities within 10km; uses city if within 5km

Advantages:

- ✓ **More granular:** Better boundaries than GADM (towns, neighborhoods)
- ✓ **Up-to-date:** OSM is actively maintained
- ✓ **Flexible:** Can query nearby cities as fallback

Disadvantages:

- × **Network dependency:** Requires Overpass API calls
- × **Rate limiting:** Subject to Overpass API limits
- × **Latency:** Network round-trip adds delay

Used By:

- API endpoint (`/api/gadm-polygon`) - **Note:** This endpoint exists but is **not the primary flow** for the interactive Locate Me page

Current State:

- API endpoint still functional but not primary path
- Used as fallback or for server-side gallery generation

✓ **Current: GeoBoundaries ADM Packs (Client-Side)**

Status: ✓ **In Use** (Primary for interactive page)

Implementation:

- **Data Source:** GeoBoundaries gbOpen boundaries
- **Storage:** `public/adm-packs/**/*.json.gz` (gzipped FeatureCollections)
- **Build Scripts:**
 - `scripts/adm/build-adm0.js` (countries)
 - `scripts/adm/build-packs.js` (ADM1-4 per country)
 - `scripts/us/build-us-place-boundaries.js` (US place/city boundaries)
- **Client Code:** `public/js/locate-me-v2.js`

How It Works:

1. **Pre-built Packs:** Boundaries pre-processed and gzipped per country/level
2. **On-Device Lookup:** Client loads relevant pack(s) and performs point-in-polygon locally
3. **Boundary Selection:**
 - **Blue (Regional):** ADM2 pack lookup
 - **Red (Local):**
 - **US:** Prefer `adm3-place` (place/city) → fallback to tract `adm3` → fallback to ADM2
 - **Other countries:** Prefer ADM3 pack → fallback to ADM2

Advantages:

- ✓ **Offline/On-device:** No network calls required
- ✓ **Fast:** Pre-processed, optimized packs
- ✓ **Privacy-preserving:** All processing client-side
- ✓ **Granular:** US place boundaries provide city-level detail

Used By:

- **Primary:** Interactive Locate Me page (`public/locate-me.html`)
- Gallery generation (`scripts/generate-locate-me-gallery.js`)

Pack Structure:

```
public/adm-packs/
├── us/
│   ├── adm3-place/           # US place/city boundaries (preferred)
│   │   ├── CA.json.gz
│   │   └── ...
│   └── adm3/                 # US tract boundaries (fallback)
│       └── ...
├── de/
│   ├── adm2.json.gz
│   └── adm3.json.gz
└── ...
```

Current Architecture Summary

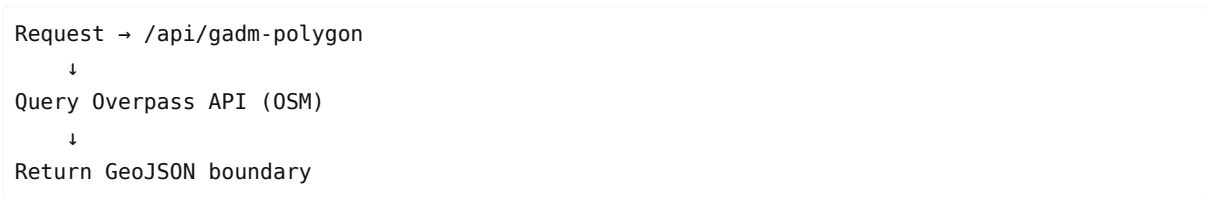
Interactive Locate Me Page (Primary Flow)



Characteristics:

- ✔ **Fully offline** after initial page load
- ✔ **No server API calls** for boundaries
- ✔ **Privacy-preserving** (no GPS sent to server)

API Endpoint (Fallback/Server-Side)



Characteristics:

- ⚠ **Network-dependent** (Overpass API)
- ⚠ **Used for gallery generation** or as fallback
- ⚠ **Not primary path** for interactive page

Strategy Comparison

Strategy	Status	Location	Network	Granularity	Speed
GeoNames Offline	✔ Current	Client	× No	City-level	⚡ Fast
GADM Shapefiles	× Deprecated	Server	✔ Yes	Regional	🐢 Slow
OSM/Overpass API	⚠ Transitional	Server	✔ Yes	Very granular	🐢 Medium
GeoBoundaries Packs	✔ Current	Client	× No	Regional/Local	⚡ Fast

Migration History

- Initial:** GADM shapefiles (server-side, GCS)
- Transition:** Added OSM/Overpass API endpoint
- Current:** GeoBoundaries ADM packs (client-side, offline)

Key Decision Points:

- **Privacy:** Moved to client-side to avoid sending GPS to server
- **Performance:** Pre-built packs faster than on-demand queries
- **Granularity:** GeoBoundaries + US place boundaries provide good detail
- **Reliability:** Offline approach avoids API rate limits/downtime

Future Considerations

Potential Improvements

1. **Population Data:** Currently estimates by summing cities-in-polygon; could use gridded sources (WorldPop, GPW)
2. **Weather:** Currently uses coarse query points; could improve with hourly series
3. **More Countries:** Add ADM packs for additional countries as needed
4. **Boundary Updates:** GeoBoundaries updates periodically; consider automated pack rebuilds

Deprecated Components

- **GADM API:** Legacy server-side approach (documented in `docs/locate-me-gadm.md`)
 - **Overpass API:** Still functional but not primary path (may be removed if fully replaced)
-

References

- **Current Implementation:** `docs/locate-me/README.md`
- **Legacy GADM:** `docs/locate-me-gadm.md`
- **Reverse Geocode API:** `api/reverse-geocode.js`
- **Boundary API:** `api/gadm-polygon.js`
- **Client Code:** `public/js/locate-me-v2.js`